# Weather Flow

Andrea Galliano 85641A

Real-Time weather telemetry via Apache Kafka

March 26, 2026

# 1. Introduction

This project is about building and implementing a ***Cloud-Native*** architecture for Real-Time weather telemetry using *Apache Kafka*. The system is designed to collect, process, and store Real-Time meteorological data from six different cities in Italy: Milan, Turin, Verona, Florence, Rome and Naples (the data is retrieved via *REST API* available on `open-meteo.com` °). In addition, a continuos stream of measurements (temperature, rainfall, wind speed...) is simulated by an external script and sent to the Kafka cluster. From there, the data is consumed in parallel by multiple microservices for specific purposes: Real-Time data visualization via a web dashboard, data storage in *NoSQL* database and alert generation for extreme weather conditions, forwarded to a smartphone via Telegram bot. The main focus is to garantee the system's scalability, reliability, criptographic security and fault-tolerance.

# 2. Architectural overview

The entire infrastructure is based on an architectural pattern with deacupled microservices. The communication is not direct, but rather mediated by a message broker, in this case Apache Kafka. This is the architectural overview of the system:

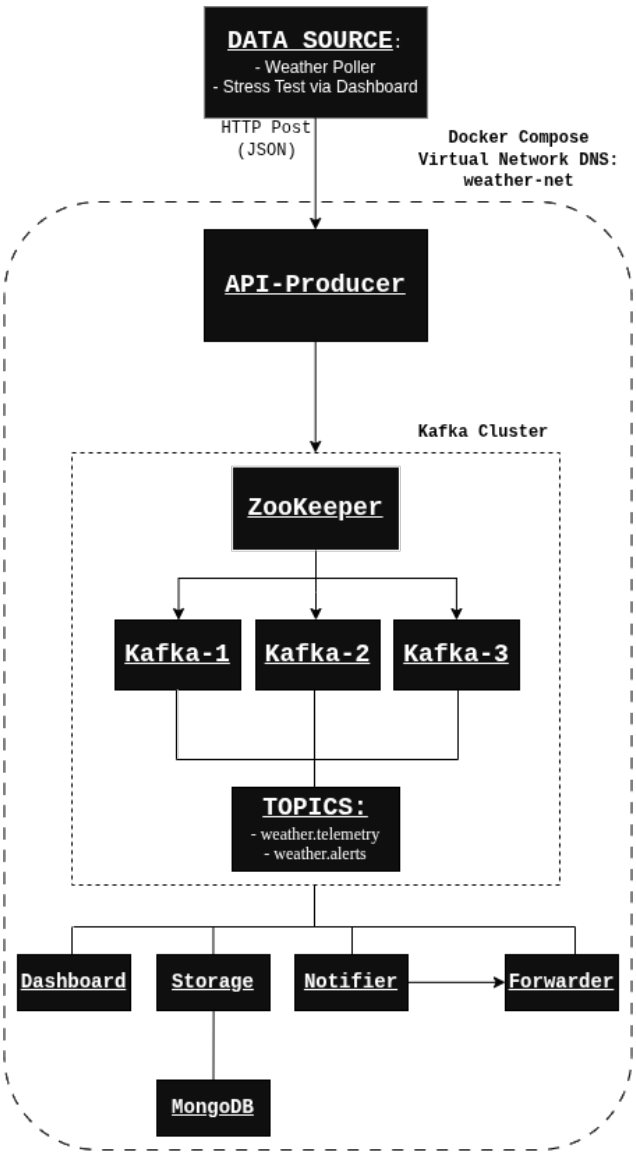


Figure 1: Architectural overview of the system.

# 3. Description of the system

## 3.1. Components

The entire system is composed of the following components:

- **Weather-Poller**: Python script that periodically retrieves weather data from `open-meteo.com` ° for the six cities and sends it to the API-Producer.
- **API-Producer**: REST application that receives weather data in JSON format and sends it to the Kafka cluster. Thanks to *HTTP POST* requests, the producer publishes the data to the main Kafka topic ( `weather.telemetry` ) instantly.
- **Dashboard**: A Real-Time web application built with ***Flask***. Besides visualizing data, its backend integrates a custom "**Stress Test**" to simulate massive data ingestion, effectively showcasing *Apache Kafka*'s ability to handle high-throughput event queues and dynamic load balancing without performance degradation.
- **Storage**: Microservice responsible for consuming the data from the Kafka topic and storing it in a ***NoSQL database*** (*MongoDB*).
- **Notifier**: Microservice that consumes the data from the Kafka topic and generates alerts for extreme weather conditions (e.g., high or low temperature, heavy rainfall, strong winds). If Notifier detects an extreme weather condition, it publishes an alert message to a specific Kafka topic ( `weather.alerts` ). Notifier is a Consumer, but also a Producer.
- **Telegram Forwarder**: Isolated microservice that only consumes alert messages from the `weather.alerts` topic and uses Telegram's API to forward notifications to the users' smartphones.

## 3.2. Streaming processing and distributed coordination

The backbone of the architecture is rapresented by a distributed messaging cluster (it is the "*Single Source of Truth*") for alla the microservices:

- **Apache Kafka**: it manages data flow and divides it in **Topics** ( `weather.telemetry` and `weather.alerts` ). Kafka brokers push data to the Consumers, but they also act as immutable registers (with high performance) for microservices that pull data autonomously and at their own rithm.
- **ZooKeeper**: It is the distributed coordinator for the Kafka cluster. It manages and keep metadata in cluster, manage brokers' registration and leader election, and it also monitors the health of the cluster. Thanks to *ZooKeeper*, the system can automatically recover from failures and maintain high availability.

## 3.3. Infrastructure as code and containerization

The entire architecture is based to be "*Cloud-Ready*", isolating processes in containers and making deployment universal and reproducible. The main tools used for this purpose are:

- **Docker**: each microservice is packeged in an isolated conainer, starting with a specific image ( `python:3.11-slim` ). This approach guaratees code portability and consistency across different environments.
- **Docker Compose**: used to orchestrate the deployment of all microservices (*Infrastructure as Code*). The `docker-compose.yml` file instances containers simultaneously, defines their dependencies and manages the network configuration. This allows for easy scaling and management of the entire system.

# 4. Non-functional requirements

Industrial standards respected:

- **Fault Tolerance**: Kafka cluster is configured with a replication factor of 3. Data is replicated accross phisically separated nodes, so, in case of a node failure, *Zookeeper* automatically promotes a backup replice (*ISR*) to a new leader whithin few milliseconds, without any event loss or interruption of the service.
- **Security**: Internal communication between microservices and Kafka cluster is encrypted using ***mTLS*** *(Mutual Transport Layer Security)*. Each Kafka broker holds a **keystore** containing its own **RSA** certificate and private key, and a truststore containing the **CA** certificate used to authenticate incoming connections. Each *Python* microservice is provided with a **PEM-encoded** certificate and private key, both signed by an internal ***CA*** *(Certificate Authority)*. During the TLS handshake, both parties present and mutually verify their certificates against the shared CA. Any connection attempt from a client not holding a valid CA-signed certificate is rejected at the transport layer, making unauthorized access to Kafka topics computationally infeasible.
- **Load Balancing**: this requirement is handled natively by Apache Kafka architecture, comining topic partitioning with the strategic use of consumer groups. Services like Notifier and Storage use static Group IDs: in case of horizontal scaling, Kafka ripartizes autonomatically topic's partitions among the replicas of the microservice, distributing the load evenly. The only exception is the **Dashboard**, which uses a randomly generated Group ID (backed by a sufficiently large keyspace) to ensure each instance is treated as an independent consumer group, allowing all instances to consume the same data in parallel without load balancing, implementing a *broadcasting pattern* suited for real-time visualization.

# 5. Showcases

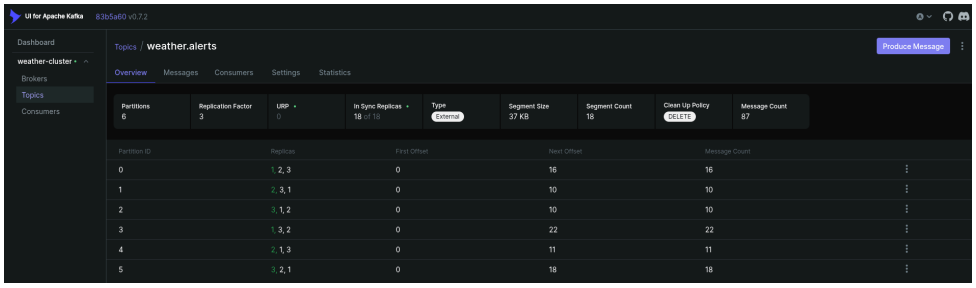


Figure 2: Load Balancing on 6 partitions of `weather.telemetry` topic, viewable thanks to Kafka-UI°.

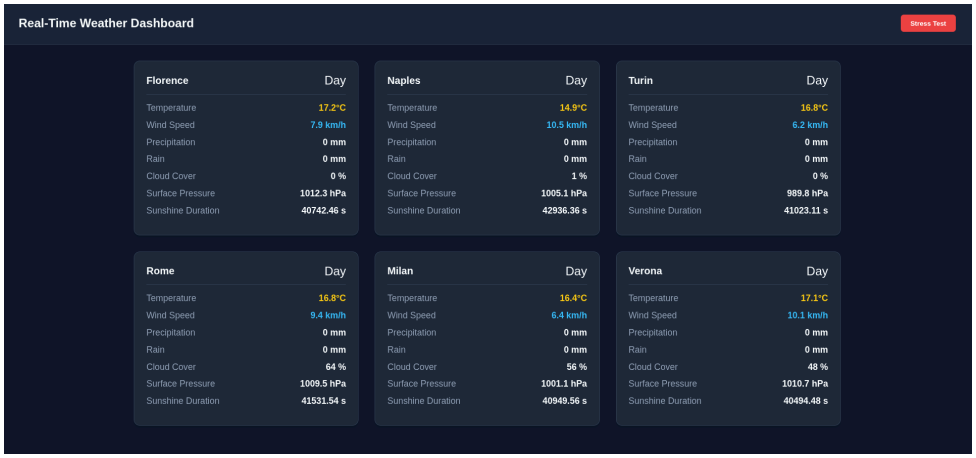


Figure 3: Frontend Dashboard with Real-Time data.

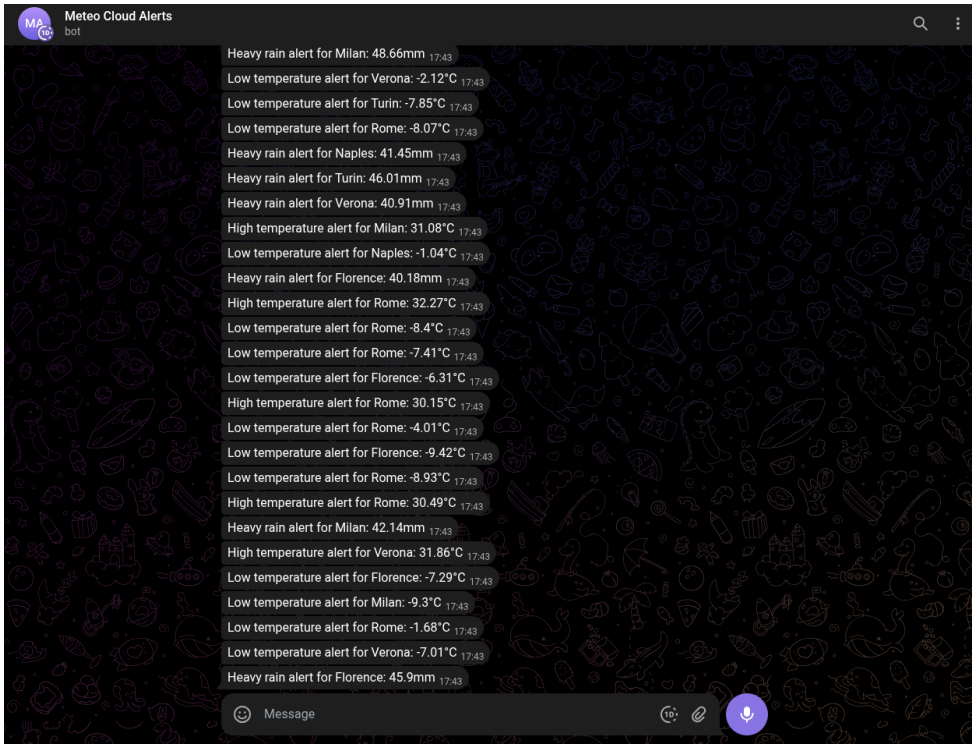


Figure 4: Telegram Forwarder chat with some weather alerts.